

```
x[20] = 1; //Invalid write as x[20] is not allocated any memory
```

Remember that here, `x[20]` being assigned a value is not a valid operation; hence, this will lead to a segmentation fault. We know that the size of `x` is just 20 and hence accessible locations are `x[0]` to `x[19]`, so `x[20]` is an invalid address, and writing to it is an invalid write.

On compiling the code and running it in Valgrind, you will get the following response:

```
==5235== Invalid write of size 4
==5235==    at 0x804853A: main (eg1.cpp:8)
==5235==    Address 0x4328078 is not stack'd, malloc'd or
(recently) free'd
```

The output indicates there is an invalid write of size 4 happening at location `0x804853A`, i.e., at Line No 8 in the `main` function of the program. In other words, it gives you the type of error and the stack trace, which gives you the location of the error.

Let us now add the following lines to our example code, to consider Scenario 3.

```
if(x[1] == 0) // x[1] is not assigned any value. Hence invalid read.
    cout<<"Hello";
```

There is an invalid read happening in the new line. Remember that we did not assign any values to the array. Therefore, `x[1]` has a garbage value and reading that is an invalid read—a memory fault. On compiling and running the program in Valgrind, you will get the following output:

```
==5244== Invalid read of size 4
==5244==    at 0x80485D7: main (eg1.cpp:9)
==5244==    Address 0x432802c is 0 bytes after a block of size 4
alloc'd
==5244==    at 0x402B87E: operator new(unsigned int) (vg_replace_
malloc.c:292)
==5244==    by 0x80485B8: main (eg1.cpp:7)
```

Note that MemCheck does not report an error when it finds uninitialised data, but reports only when uninitialised data is used in the program.

You can explore the other scenarios in a similar way, if you are interested. Valgrind effectively finds unpaired calls to `new/malloc` and `delete/free`, invalid memory operations like read and write, and detects system calls with inadequate read-write parameters.

Installing the Valgrind framework

Valgrind is available from the Ubuntu repositories. You can check the repositories for other distributions, or directly install the program using the source from <http://valgrind.org/downloads/>

for the latest version, after which, follow the steps given below:

```
$ bzcat2 -d valgrind-XYZ.tar.bz2
$ tar -xf valgrind-XYZ.tar
$ ./configure
$ make
$ make install
```

Using Valgrind during software development can improve the quality of the software being developed. To get more information about Valgrind, please refer to the home page, <http://valgrind.org/> 

References

- [1] <http://www.cprogramming.com/debugging/valgrind.html>
- [2] <http://valgrind.org/>
- [3] <http://valgrind.org/downloads/>
- [4] <http://cs.ecs.baylor.edu/~donahoo/tools/valgrind/messages.html>
- [5] <http://www.around.com/ariane.html>
- [6] <http://valgrind.org/docs/manual/faq.html#faq.defrost>

By: Sreepriya C

The author is a FOSS enthusiast and is also interested in cyber security. Currently, she works on mastering various tools for debugging and disassembling binaries for reverse engineering.



Online Training Programmes

Now access world class training courses on Linux system programming from the comforts of your home

Courses Offered

- Linux Programming and Device Drivers
- Linux Debugging Expert
- Adv. C Programming



Visit www.vedasol.org or call 0805560595 for more details



Over 1000 participants from over 6 countries
Over 10 MINs including Logixmat, Broadcom, Xilinx...

201, Prashanthi Ram Towers, Sarathi Studio Line, Amrpet, Hyderabad-500075. Ph: 040-66100255